Query 9:

```sql
-- Query 9: Calculate the average length of stay for each critical care unit type
-- Uses `AVG()` to calculate the average stay duration, `GROUP BY` to separate unit types, and `WHERE` to filter critical care units and incomplete stays.
      You, yesterday • Add SQL queries 9-12 …
SELECT
    unit_type, -- display the critical care unit type
    JUSTIFY_HOURS(AVG(exit_datetime - entry_datetime)) AS average_length_of_stay -- calculates and formats the average length of the stay
FROM UnitStays -- retrieves unit stay records
WHERE unit_type IN ('MICU', 'SICU', 'ICU', 'CCU') -- includes only critical care units (MICU, SICU, ICU, CCU)
    AND exit_datetime IS NOT NULL -- excludes any stays that have not yet ended
GROUP BY unit_type -- calculates a seperate ADD
ORDER BY unit_type; -- displays the results in alphabetical order by unit type
```

```
 unit_type | average_length_of_stay
-----------+------------------------
 CCU       | 7 days 17:15:00
 ICU       | 6 days 13:36:00
 MICU      | 8 days 15:00:00
 SICU      | 5 days 23:00:00
(4 rows)
```

The query uses WHERE to select completed stays in critical care units. It subtracts the entry time from the exit time to calculate each stay's duration, then uses AVG() and GROUP BY to find the average for each unit type. JUSTIFY_HOURS() formats the results in days and hours.

Query 10:

```sql
-- Query 10: Patients who have underwent surgey before entering the ICU/CCU unit
-- Uses `JOIN` to connect patients, admissions, surgeries and unit stays, and `WHERE` to filter surgeries performed before ICU/CCU entry

SELECT
    p.first_name, -- display patient's first name
    p.last_name, -- display patient's last name
    s.procedure_datetime, -- display surgery date and time
    u.unit_type, -- display critical care unit type
    u.entry_datetime AS unit_entry_time -- display unit entry time
FROM Patients p -- retrieve patient information
JOIN Admissions a ON p.patient_id = a.patient_id -- matches patients to admissions
JOIN SurgicalProcedures s ON a.admission_id = s.admission_id -- to find surgeries during admission
JOIN UnitStays u ON a.admission_id = u.admission_id -- find unit stays during the same admission
WHERE u.unit_type IN ('ICU', 'CCU') -- include only ICU and CCU
    AND s.procedure_datetime < u.entry_datetime -- ensure surgery occurred before unit entry
ORDER BY p.last_name, p.first_name, s.procedure_datetime; -- sort by patient and surgery date
```

first_name	last_name	procedure_datetime	unit_type	unit_entry_time
Amira	Bennett	2025-03-24 13:00:00	ICU	2025-03-24 15:00:00
Omar	Khan	2025-08-21 21:00:00	ICU	2025-08-21 23:00:00
Felix	Lavoie	2026-02-17 21:00:00	ICU	2026-02-17 23:00:00
Lucas	Martin	2025-11-19 21:00:00	ICU	2025-11-19 23:00:00
Julien	Mercier	2025-05-23 21:00:00	ICU	2025-05-23 23:00:00
Camille	Roy	2025-08-06 19:00:00	ICU	2025-08-06 21:00:00
Camille	Roy	2025-08-06 19:00:00	CCU	2025-08-08 20:30:00
Daniel	Tremblay	2025-02-07 19:00:00	ICU	2025-02-07 21:00:00
Daniel	Tremblay	2025-02-07 19:00:00	CCU	2025-02-12 08:30:00

The query uses JOIN to connect patients, admissions, surgeries and unit stays. The WHERE clause filters ICU/CCU stays where the surgery happened before the patient entered the unit. ORDER BY sorts the results alphabetically by patient name then by surgery date.

Query 11:

```sql
-- Query 11: Count the radiology examinations performed for each patient across all admissions
-- Uses `LEFT JOIN` to include all patients and `COUNT()` with `GROUP BY` to total radiology examinations per patient

SELECT
    p.first_name, -- display patient's first name
    p.last_name, -- display patient's last name
    COUNT(r.radiology_exam_id) AS total_examinations -- count radiology exams, including zero
FROM Patients p -- start with all patients
LEFT JOIN Admissions a ON p.patient_id = a.patient_id -- include patients without admissions
LEFT JOIN RadiologyExaminations r ON a.admission_id = r.admission_id -- include admissions without radiology exams
GROUP BY p.patient_id, p.first_name, p.last_name -- calculate one total per patient
ORDER BY p.last_name, p.first_name; -- sort results alphabetically
```

first_name	last_name	total_examinations
Amira	Bennett	3
Mila	Biron	0
Chloe	Bouchard	0
Maya	Chen	2
Leo	Clarke	0
Mateo	Costa	2
Elise	Dubois	2
Ethan	Foster	1
Ines	Gagnon	2
Adele	Girard	1
Leila	Haddad	2
Omar	Khan	2
Zoe	Lambert	0
Sofia	Laurent	2
Felix	Lavoie	1
Rania	Mansour	0
Lucas	Martin	1
Julien	Mercier	1
Victor	Moreau	1
Nina	Nguyen	0
Noah	Patel	1
Theo	Pelletier	0
Hugo	Poirier	0
Adam	Reed	1
Jade	Rossi	0
Camille	Roy	1
Samir	Salem	0
Daniel	Tremblay	4

(28 rows)

The query uses LEFT JOIN to connect patients, admissions and radiology examinations while including patients with no exams. It then uses COUNT() and GROUP BY to calculate the total examinations per patient across all admissions, and ORDER BY to sort the results alphabetically.

Query 12:

```sql
-- Query 12: Find all patients whose discharge-summary clinical note contains the keyword 'recovery'
-- Uses `JOIN` to connect patients to clinical notes, `WHERE` with `ILIKE` to find discharge summaries containing “recovery” for case-insensitive matching
-- and `DISTINCT` to avoid duplicates

SELECT DISTINCT
    p.first_name, -- display patient's first name
    p.last_name -- display patient's last name
FROM Patients p -- retrieve patient information
JOIN Admissions a ON p.patient_id = a.patient_id -- match patients to admissions
JOIN ClinicalNotes n ON a.admission_id = n.admission_id -- retrieve clinical notes for each admission
WHERE n.note_type = 'Discharge Summary' -- include only discharge-summary notes
    AND n.note_content ILIKE '%recovery%' -- search for "recovery" regardless of capitalization
ORDER BY p.last_name, p.first_name; -- sort patients alphabetically
```

```
 first_name | last_name
------------+-----------
 Amira      | Bennett
 Maya       | Chen
 Mateo      | Costa
 Elise      | Dubois
 Ethan      | Foster
 Ines       | Gagnon
 Adele      | Girard
 Julien     | Mercier
 Noah       | Patel
 Adam       | Reed
 Camille    | Roy
 Daniel     | Tremblay
(12 rows)
```

The query uses JOIN to connect patients, admissions, and clinical notes. The WHERE clause filters discharge summaries that contain “recovery” using ILIKE for case-insensitive matching. DISTINCT removes any duplicate patients. ORDER BY sorts the results alphabetically.